

CARET: A Spec-Driven Development Framework for Contract-Aware Repository Editing and Governed Agentic Code Generation

Ciprian Paduraru¹ Petru-Liviu Bouruc¹ Alin Stefanescu^{1,2}

¹University of Bucharest, Romania

²Institute for Logic and Data Science, Romania

30th International Conference on Knowledge-Based and
Intelligent Information & Engineering Systems (KES 2026)

Artifact: github.com/unibuc-cs/caret-agent-bench

The one-slide summary

Problem

LLM agents that edit repositories can produce a patch that **passes the tests** and still **breaks the rules**: authorization boundaries, logging and redaction policy, schema-evolution procedure, data-handling obligations.

Idea

Make governance a **machine-readable contract** — the **Architecture Card** — injected into the agent workflow, enforced during execution, and checked at a verification gate over *both* the patch and the execution trace.

Result (hard split of CARET-Bench)

Compliant success **0.58 → 0.93** at unchanged functional success (0.93); policy violations down **86%**. An ablation shows the **gate alone is not enough** — repair support is essential.

PART 1

Motivation

Repository-scale agents: the unit of analysis has changed

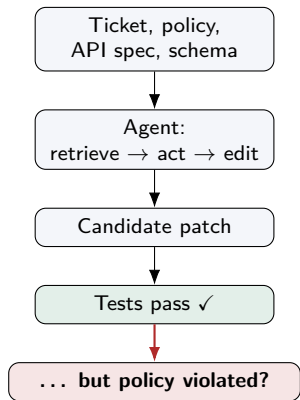
Agents now perform **workflows**, not isolated tasks:

- retrieve tickets, docs, schemas, policies
- inspect and edit many source files
- invoke tools (registries, migrations, deploys)

SWE-bench, SWE-agent, AutoCodeRover, Agentless, DevOps-Gym, all of them grade the patch, not the run...

Consequence

The object to be judged is not only the final **diff**, but the **sequence of retrievals, actions and checks** that produced it.



Two gaps

1. Functional success $\neq$ governance compliance

An agent can satisfy every task-level requirement while violating authorization boundaries, logging/redaction rules, schema-evolution procedures, or data-handling obligations.

Tests measure *what the code does*, not *whether the agent followed the process*..

2. Tool-using workflows lack explicit decision provenance

Sensitive actions *should* be traceable to approved evidence (a ticket, a schema version, a policy) but that grounding is usually absent from the execution record.

Two gaps

1. Functional success $\neq$ governance compliance

An agent can satisfy every task-level requirement while violating authorization boundaries, logging/redaction rules, schema-evolution procedures, or data-handling obligations.

Tests measure *what the code does*, not *whether the agent followed the process*..

2. Tool-using workflows lack explicit decision provenance

Sensitive actions *should* be traceable to approved evidence (a ticket, a schema version, a policy) but that grounding is usually absent from the execution record.

Both gaps are about the *process*, so the fix has to live in the process.

A concrete request

The task (small)

“Add a field to the user-profile endpoint and update the response format.”

A few lines of code. Easy to test. Easy to get *functionally* right.

The rules that also apply (invisible to tests)

- the new field must not expose PII
- a public API change must ship with test updates
- deployment is restricted to one environment tier
- consent must be verified *before* any PII read

CARET turns these from informal review criteria into *executable workflow conditions*.

1. **Framing:** repository-scale agentic code generation as *Spec-Driven Development*, treating functional success and success-under-governance as *distinct* outcomes.
2. **Contract model:** CARET formalizes patch constraints, runtime checks, tool-use bounds and trace requirements in one artifact.
3. **Execution-level enforcement:** mediated tool access, structured trace logging, verification against trace-aware contracts.
4. **Prototype:** Architecture Cards + trace ledger + verification gate + bounded repair.
5. **Artifact:** open-source implementation and **CARET-Bench**, a benchmark for governance-sensitive repository editing.

<https://github.com/unibuc-cs/caret-agent-bench>

PART 2

The CARET framework

$$AC = \langle \Sigma, \Phi, \mathbb{M}, \tau \rangle$$

Σ — static invariants

Evaluated over the candidate patch C .
"The finished code must satisfy this property" e.g. authorization wrapper required; no secret literals.

Φ — runtime constraints

Evaluated over observations from tests / monitors.
"The tests must pass" e.g. `p95_latency_ms <= 200`.

$\mathbb{M}$ — capability bounds

Evaluated over attempted tool actions in environment tier E .
"You may use only these tools, and write only these files, in this tier" e.g. production is `READ_ONLY`; `DROP_TABLE` blocked.

τ — trace contracts

Evaluated over the ordered trace ledger H .
"You must have done X before you did Y" e.g. schema lookup *must precede* a migration.

Task evidence (tickets, repo docs, API specs, schemas, approved policies) forms the *Semantic Corpus* SC . Each request yields a patch C and a trace ledger H .

The Architecture Card

```
meta: {version: "0.2", governance_tier: "strict"}

static_invariants:  # Sigma  -- checked on the patch C
  - id: "SIGMA_AUTHZ_WRAPPER_REQUIRED"
    enforcement: "AST_PATTERN"

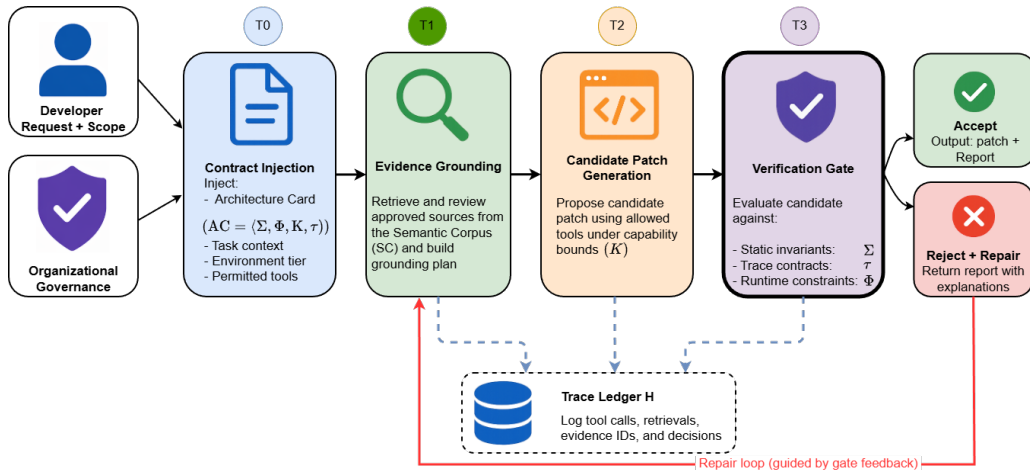
runtime_constraints: # Phi    -- checked via tests / monitors
  - id: "PHI_LATENCY_BUDGET"
    constraint: "p95_latency_ms <= 200"
    action_on_breach: "FAIL_GATE"

capabilities:       # K      -- enforced by tool mediation, per tier
  production:
    mode: "READ_ONLY"
    blocked_actions: ["DELETE", "DROP_TABLE", "MIGRATE_SCHEMA"]

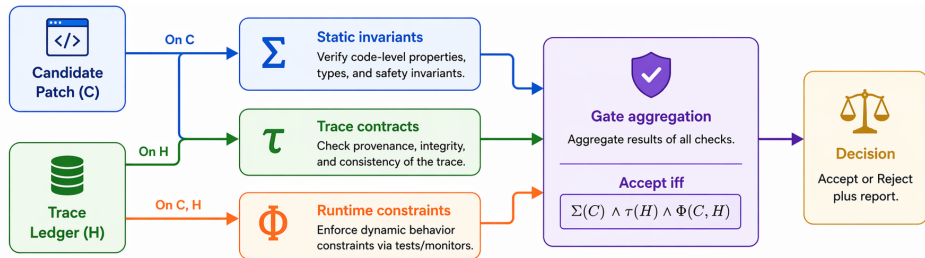
trace_contracts:    # Tau    -- checked on the trace ledger H
  - id: "TAU_SCHEMA_LOOKUP_PRECEDES_MIGRATION"
    trigger: "event_type=decision AND decision.action=MIGRATE_SCHEMA"
    must_precede: "SchemaRegistry.getLatestMetadata(table)"
```

Injected at start. $\mathbb{M}$ is enforced *during* execution; Σ , τ , Φ are evaluated at the *Verification Gate*.

Request lifecycle



The verification gate



$$R(AC, C, H; E) = \begin{cases} \text{accept} & \text{if } \forall \sigma \in AC.\Sigma : \sigma(C) \wedge \forall \tau_i \in AC.\tau : \tau_i(H) \wedge \\ & \forall \phi \in AC.\Phi : \phi(C, H; E) \\ \text{reject+repair} & \text{otherwise} \end{cases}$$

The trace ledger H

```
{
  "run_id": "uuid",
  "environment": {"tier": "staging|production"},
  "events": [{
    "stage": "T1|T2|T3",
    "event_type": "retrieval|tool_call|decision",
    "retrieval": {"source": "schema_registry",
                  "id": "doc_id", "hash": ".."},
    "tool_call": {
      "name": "SchemaRegistry.getLatestMetadata",
      "args": {"table": "CustomerProfile"},
      "result": {"status": "ok", "hash": ".."}},
    "decision": {"action": "MIGRATE_SCHEMA",
                  "rationale_refs": ["ticket_id"]}
  ]},
  "outputs": {"patch_id": "sha", "tests": {...}}
}
```

Why it matters

Events are **typed** (retrieval / tool_call / decision) and **staged** (T_1 – T_3), with evidence identifiers and hashes.

What it enables

Trace contracts $\tau(H)$ become checkable through explicit **witnesses** and **ordering** constraints:

“which information objects were consulted before this high-impact action?”

Assumes mediated tool access and integrity-protected event capture.

Reusable policy templates

Template	Σ (patch)	τ (trace)	$\mathfrak{M}$ (tools)	Φ (runtime)
PII / Consent	mask PII in logs	consent precedes PII read	block PII export	residency checks
Secrets	forbid literals	secret lookup precedes connect	restrict env. access	rotation check
Schema change	forbid raw edits	schema lookup precedes migrate	block migrate (prod)	migration canaries
AuthZ wrapper	enforce wrappers	policy lookup precedes change	restrict role tools	audit-log assertion
Outbound net	forbid raw HTTP	justification cites allowlist	egress allowlist	anomaly monitors

PART 3

Evaluation

Benchmark — 12 scenario packages

Easy: 9 scenarios, 108 task instances
localized changes, ~1–3 files, limited cross-file
interaction, weak policy coupling

Hard: 3 scenarios, 45 task instances
~3–15 files, richer structure, stricter tool
constraints, requirements depending on evidence
grounding and traceable justification

Derived from the SWE-bench setting. Each
scenario = repository state + task description +
Architecture Card + executable validation setup.
3 runs per task and mode; results are averages.

Modes: increasing structure of control

tests_only — one shot, tests are the only
gate

baseline — same gate, plus a retry loop fed
by test-failure feedback

CARET — card injection + mediated tools +
trace-aware verification + bounded repair

CARET_no_operator — *ablation*: gate active,
deterministic compliance operator disabled

Two outcome notions

Success: task-level acceptance criteria met.

Compliant success: accepted *and* no residual
policy violation.

Mode	Success	Compliant	Avg. Viol.	Avg. Rej.	Avg. Retry
tests_only	0.95	0.93	0.02	0.00	0.00
baseline	0.95	0.92	0.03	0.00	0.00
CARET	1.00	1.00	0.00	0.28	0.23

9 scenarios, 108 task instances, averaged over 3 runs.

- The easy split is **already favourable to every mode** — test-gated generation performs strongly on localized edits.
- CARET still reaches 1.00 / 1.00 and eliminates residual violations.
- Cost of stricter governance: **0.28 rejections and 0.23 retries per run.**

Mode	Success	Compliant	Avg. Viol.	Avg. Rej.	Avg. Retry
tests_only	0.89	0.49	0.58	0.00	0.00
baseline	0.93	0.58	0.47	0.00	0.24
CARET_no_operator	0.27	0.27	1.36	2.24	1.51
CARET	0.93	0.93	0.07	0.22	0.16

3 scenarios, 45 task instances, averaged over 3 runs.

- Baselines **pass the tasks but leave violations unresolved**: 0.93 success vs. only 0.58 compliant.
- CARET keeps the **same functional success** (0.93) and raises compliant success to **0.93**.
- Average policy violations: $0.47 \rightarrow 0.07$, an **86% reduction**, for 0.22 rejections and 0.16 retries per run.

CARET_no_operator

Verification gate active, **deterministic compliance operator disabled**.

- functional & compliant success: **0.27**
- average policy violations: **1.36**
- rejections: 2.24 retries: 1.51

Reading

The gate **detects** violations reliably. Without targeted repair, it cannot **convert a rejection signal into a compliant solution**: the loop rejects, retries, and stalls.

Design consequence

Governance checking is *necessary but not sufficient*. Enforcement must ship with a repair operator that can insert required reads, trace witnesses, or test touches before re-verification.

Simple tasks

Standard test-gated generation **already performs strongly**.

Governance-aware execution adds overhead with limited measurable benefit.

Hard, multi-file, governance-sensitive tasks

Explicit governance representation **substantially improves compliant outcomes** without reducing the strongest observed functional success.

Limitations

- CARET-Bench is synthetic and compact and not production scale or organizational complexity.
- The easy split predates the ablation, so `CARET_no_operator` covers only the hard split.
- *Policy violations* is a lightweight post-run proxy, not a formal proof obligation.

PART 4

Conclusion

What the evaluation shows

- Value depends on task difficulty: near-free on easy tasks, **+35 points** of compliant success on hard ones.
- Functional performance is preserved, residual violations drop **86%**.
- A verification gate **alone** is insufficient — repair support is essential.

Contribution

CARET's contribution is *control over how project information becomes action* when the workflow is governance-sensitive.

Future work

Architecture Cards for coordinated **multi-agent** workflows; stronger **trace-contract semantics**; combining hand-authored policies with **learned or synthesized** templates; evaluation on **large production repositories** with legacy code and long histories.

Thank you

Questions?

Artifact — prototype implementation + CARET-Bench
<https://github.com/unibuc-cs/caret-agent-bench>

Ciprian Paduraru · Petru-Liviu Bouruc · Alin Stefanescu

University of Bucharest · Institute for Logic and Data Science

Partially funded by Adobe Romania and by the “Romanian Hub for Artificial Intelligence (HRIA)” project,
Smart Growth, Digitization and Financial Instruments Programme 2021–2027, MySMIS No. 351416.

BACKUP SLIDES

Additional material

```
function VerifyAndGate(patch C, trace H, card AC) -> (decision, report):
  report := {}
  # 1) Static invariants (Sigma)
  for sigma in AC.static_invariants:
    ok, detail := CheckStaticInvariant(sigma, C)
    if not ok:
      report.add({"type": "SIGMA_FAIL", "id": sigma.id, "detail": detail})
      return ("reject+repair", report)

  # 2) Trace contracts (Tau)
  for tau in AC.trace_contracts:
    triggered := MatchTrigger(tau.trigger, H)
    if triggered:
      ok, witness := CheckMustPrecede(tau.must_precede, H, triggered)
      if not ok:
        report.add({"type": "TAU_FAIL", "id": tau.id, "witness": witness})
        return ("reject+repair", report)

  # 3) Runtime constraints (Phi) - delegated to tests/monitors
  ok, phi_detail := CheckRuntimeConstraints(AC.runtime_constraints)
  if not ok:
    report.add({"type": "PHI_FAIL", "detail": phi_detail})
    return ("reject+repair", report)
```

Backup — step descriptors (structured tool gating)

Preconditions and permissions are made explicit *before* tool execution, which is how $\mathfrak{M}$ is enforced at runtime.

```
{
  "step_id": "S2_VERIFY_SCHEMA",
  "step_type": "tool",
  "preconditions": [
    {"context": "environment.tier", "constraints": {"const": "staging"}},
    {"actor": "agent_service_account", "required_permission": "SCHEMA_READ"}
  ],
  "actions": [
    {"tool": "SchemaRegistry.getLatestMetadata", "next_step": "S3_GENERATE_MIGRATION"}
  ]
}
```

Existing enforcement scopes

- *Test-driven / verifier-guided*: constrain functional behaviour of the artifact.
- *Formal verification*: stronger guarantees, needs a formal model rarely available in ordinary repo editing.
- *Constraint-guided generation*: restricts outputs, but constraints stay prompt- or task-level.
- *Secure-code benchmarks*: measure the artifact, not the workflow.

CARET

Governance as an **execution contract** spanning

- patch invariants,
- runtime checks,
- tool permissions,
- trace requirements.

The novelty is not a new verifier — it is the *integration* of these controls into one card that is injected, enforced, logged, and repaired against.

Backup — PII policy template

```
template_id: "PII_STRICT_V1"

static_invariants:
  - id: "SIGMA_MASK_PII_IN_LOGS"
    enforcement: "AST_PATTERN"
    patterns: ["Logger.*(Mask(PII_FIELD))"]

capabilities:
  production:
    blocked_actions: ["EXPORT_PII", "SEND_TO_PUBLIC_NET"]

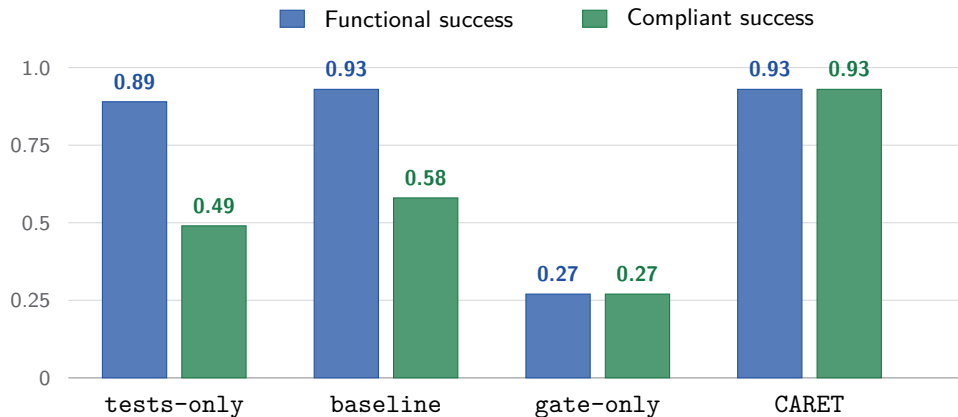
trace_contracts:
  - id: "TAU_CONSENT_PRECEDES_PII"
    trigger: "read(field) WHERE field.tags CONTAINS 'PII'"
    must_precede: "ConsentManager.verifyStatus(user_id)"
    failure_action: "REJECT_WITH_EXPLANATION"
```

Templates are controlled artifacts, instantiated with repository-specific parameters (service name, endpoints, allowed tools, environment tier).

- **Success** — the candidate satisfies the task-level acceptance criteria.
- **Compliant success** — accepted *and* no remaining policy violation under the post-run checks.
- **Average policy violations** — post-run checks over the final candidate: PII leakage, missing required reads before sensitive edits, missing test updates for public API changes.
- **Average rejections** — gate rejections per run.
- **Average retries** — repair iterations per run.
- **Unresolved rejections** — rejections never repaired into acceptance.
- **Agent error rate** — runs terminating in an agent-level failure.

In the current prototype, policy violations are lightweight post-run checks over the final candidate rather than formal proof obligations.

Hard split, visually



The gap between the two bars is exactly what CARET closes.